VESPER — Guía de presentación y defensa individual

La presentación del equipo dura **15 minutos**, seguida por **5 minutos de preguntas**. La pauta asigna el 40% de esta evaluación al encargo y el 60% a la presentación; cada integrante debe poder explicar el código y las pruebas. Completar los nombres de los integrantes y repartir los bloques antes de ensayar.

Esta guía es un recorrido de demostración. Los resultados de las pruebas y del build se deben obtener de la ejecución real del proyecto, sin anunciar porcentajes o cantidades que no se hayan comprobado.

Preparación

- Instalar con `npm ci`, ejecutar con `npm run dev` y abrir la dirección que muestra Vite. Para la suite usar `npm test`; para comprobar la compilación, `npm run build`. Dejar a mano el editor, la terminal y el informe de testing. Jasmine/Karma requiere Chrome o Chromium; `CHROME_BIN` permite indicar su ejecutable.
- Las rutas utilizan hash: abrir, por ejemplo, `/#/productos` o `/#/admin`. `src/main.jsx` monta `HashRouter` y `src/App.jsx` declara las vistas.
- Utilizar datos ficticios. La persistencia y los pagos son simulados; no ingresar contraseñas o información personal real.
- Revisar que el catálogo tenga stock y que haya un producto con descuento. Tener preparado un carrito de una o dos unidades.
- Tener disponibles las cuentas `admin@vesper.cl` / `admin123` y `usuario@vesper.cl` / `user123`.
- Guardar un resultado reciente de las pruebas como respaldo y ejecutar la suite durante la demostración si el tiempo lo permite.
- Comprobar escritorio, tableta y móvil. Para el ensayo pueden usarse, por ejemplo, anchos de 1440, 768 y 390 píxeles; registrar solamente los tamaños que efectivamente se revisen.

Guion de 15 minutos

Tiempo	Demostración	Explicación técnica
0:00–1:00	Presentar VESPER, público objetivo y cambio desde la primera entrega.	La versión previa usaba páginas HTML y manipulación directa del DOM. Esta entrega organiza la interfaz en React y separa vistas de reglas y persistencia.
1:00–3:00	Recorrer catálogo público, búsqueda, categoría, ofertas y detalle.	Mostrar una lista que se genera a partir de datos y un componente reutilizable. Explicar qué recibe por props y qué acción comunica mediante un callback.
3:00–4:30	Cambiar el ancho de la pantalla, abrir el menú y revisar un formulario o tabla.	Identificar en el código clases reales de Bootstrap: rejilla, columnas, utilidades y contenedores responsivos. Explicar el cambio visible, no solo nombrar Bootstrap.
4:30–7:00	Agregar al carrito, cambiar cantidad, ir al checkout, simular rechazo y reintentar con aprobación.	El rechazo conserva carrito y stock. La aprobación valida, crea pedido, descuenta stock y vacía el carrito. Mostrar el resumen y explicar el cálculo de subtotal más entrega.

Tiempo	Demostración	Explicación técnica
7:00–9:00	Iniciar sesión como administrador; crear o editar un producto y categoría; consultar pedidos y reportes.	Los mantenedores usan operaciones CRUD del servicio. Las vistas administrativas y las operaciones comprueban el rol; son controles de una simulación local.
9:00–10:30	Mostrar el árbol de `src`, datos iniciales, servicio y contexto.	Explicar `src/data/seed.js`, `src/services/store.js` y `src/context/ShopContext.jsx`; distinguir datos iniciales, persistencia, snapshot y renderizado.
10:30–13:30	Ejecutar las pruebas y abrir ejemplos del código de los casos.	Presentar al menos 10 casos correctos, agrupados por renderizado, props, estado, eventos y reglas del servicio. Mostrar qué se prepara, qué acción se realiza y qué resultado se verifica.
13:30–15:00	Mostrar ERS V2, informe de testing y entregables.	Relacionar un requisito con el comportamiento y su prueba. Explicar los límites: datos por navegador, pago simulado y backend pendiente.

Si el equipo tiene tres integrantes, una distribución posible es tienda y componentes; compra y persistencia; administración y pruebas. Todos deben estudiar los tres bloques para responder preguntas individuales.

Ejemplos de explicación con el código abierto

1. Props y responsabilidad de un componente

Abrir `src/components/ProductCard.jsx` y su uso en `src/components/ProductList.jsx`. Identificar `product` y `onAdd` como props. La tarjeta invoca `onAdd(product.id)` y la página del catálogo conecta ese callback con `store.addToCart`. Explicar:

> “La tarjeta presenta un producto recibido por props. No necesita saber dónde se guarda el carrito. Cuando se pulsa el botón comunica una acción y la operación de compra se resuelve en el servicio. Así podemos reutilizar la tarjeta y probar qué muestra con datos conocidos.”

Mostrar la línea de la lista que asigna una `key` basada en el identificador. La clave ayuda a React a reconocer cada elemento cuando cambia la lista; no debe confundirse con una prop de negocio.

2. Estado de un formulario o filtro

Abrir `CheckoutPage` en `src/pages/CartPages.jsx` o `ContactPage` en `src/pages/ContactPage.jsx`. Señalar `data` en `useState`, el atributo `value` de los campos y el controlador `onChange`. Escribir en la interfaz y relacionar la acción con esas líneas:

> “El valor se conserva en estado React. El evento actualiza ese estado y React vuelve a representar el resultado. En el envío, el formulario valida y llama al servicio; si hay un error, se muestra mediante un estado de mensaje.”

Distinguir estado local de pantalla, como datos del formulario o mensajes, del estado compartido, como productos, sesión y carrito. En `CatalogPage.jsx`, la búsqueda y categoría se conservan en parámetros de URL mediante `useSearchParams`, lo que permite conservar y compartir el filtro; no se presentan como otro `useState`.

3. Persistencia y suscripción

Abrir `src/services/store.js` y `src/context/ShopContext.jsx`. Seguir una operación real, por ejemplo `addToCart` o `saveProduct`:

1. La interfaz invoca la operación con los datos necesarios.
2. El servicio comprueba reglas y permisos aplicables.
3. Se actualiza el estado y se guarda en el almacenamiento del navegador.
4. Se notifica a los suscriptores.
5. `useSyncExternalStore` recibe el snapshot actualizado y React actualiza las vistas que lo usan.

Explicar por qué el snapshot debe conservar su referencia cuando no hubo cambios: React debe poder distinguir el mismo estado de una nueva actualización. Recargar muestra que el carrito o producto se recupera del almacenamiento; no demuestra sincronización con un servidor.

4. Proceso de compra

Mostrar el pedido creado y la operación `checkout`. Explicar que el pedido guarda una copia del nombre y precio de cada ítem para que una edición posterior del catálogo no cambie el resumen de una compra previa.

Demostrar primero el rechazo. Antes y después, el carrito conserva sus líneas y no aparece un pedido nuevo. Después, elegir aprobación y verificar el resultado. Esto permite explicar una regla observable de negocio y una de sus pruebas, sin depender de una pasarela externa.

5. Responsive con Bootstrap

Abrir `ProductList.jsx` y señalar `row g-4` y `col-12 col-md-6 col-lg-4`: las tarjetas ocupan una columna en móvil, dos desde el tamaño medio y tres desde el grande. En `Layout.jsx`, `navbar-expand-xl` y el estado `open` controlan el menú adaptable. Revisar navegación, tabla administrativa y formulario además de las tarjetas: un sitio responsivo necesita que sus acciones sean utilizables.

Cómo explicar Jasmine y Karma

Jasmine permite describir casos con `describe` e `it`, preparar datos y verificar resultados con `expect`; los spies registran si se llamó una función y con qué argumentos. **Karma** inicia el navegador y coordina la ejecución de la suite. React se monta en un DOM de prueba y se simulan acciones para observar sus resultados.

Abrir `karma.conf.cjs`, `tests/components.spec.jsx`, `tests/admin.spec.jsx`, `tests/store.spec.js` y `tests/helpers.jsx`. La configuración procesa JSX con Babel/webpack, aplica instrumentación de cobertura y ejecuta Chromium headless. Elegir casos presentes en la suite final y explicarlos con esta secuencia:

1. **Preparar:** crear el componente con props conocidas, almacenamiento aislado o un estado inicial controlado.
2. **Actuar:** renderizar, escribir, hacer clic o invocar una operación.
3. **Verificar:** comprobar el texto, la aparición de un mensaje, una llamada o el cambio de estado esperado.
4. **Limpiar:** desmontar componentes y descartar datos para evitar que un caso contamine al siguiente.

Conviene demostrar ejemplos de estas familias, siempre usando los casos efectivamente implementados:

Familia	Ejemplo de comportamiento a localizar en la suite
Renderizado	Una tarjeta muestra nombre y precio del producto recibido.
Listas	Se muestran todos los elementos del conjunto de prueba o un estado vacío.
Condiciones	Se presenta un error solamente cuando corresponde o se desactiva una acción sin stock.
Props y eventos	Un clic invoca el callback esperado con los datos correctos.
Estado	Escribir o seleccionar una opción cambia el filtro o el formulario.
Carrito	La cantidad respeta límites y el total se recalcula.
Validación	Un dato inválido o un correo duplicado se rechaza con un mensaje.
Pago	El rechazo no altera carrito/pedidos/stock; la aprobación realiza los cambios esperados.
Acceso	Un usuario sin rol administrador no puede modificar el catálogo.
Persistencia	Una nueva instancia del servicio recupera los datos guardados en el almacenamiento de prueba.

En `tests/components.spec.jsx`, el caso “ejecuta una sola vez el callback al agregar el producto” utiliza `jasmine.createSpy` y `toHaveBeenCalledOnceWith(product.id)` para comprobar la interacción real del botón. El caso “actualiza el DOM cuando cambia el estado de un formulario controlado” escribe un nombre y comprueba el valor del campo y del `output`. En `tests/store.spec.js`, “conserva carrito, stock y pedidos al rechazar un pago” compara el estado antes y después de una operación fallida.

Un mock o almacenamiento en memoria sustituye una dependencia para controlar las entradas y aislar cada caso. Un spy permite comprobar una interacción. Ninguno equivale a probar un sistema remoto real.

Al leer los resultados, distinguir:

- **Casos aprobados:** escenarios escritos que se ejecutaron sin fallar.
- **Cobertura funcional:** requisitos y comportamientos comprobados por esos escenarios.
- **Cobertura de código:** líneas, ramas, funciones o sentencias ejecutadas sobre los archivos instrumentados, solamente si existe un reporte medido.

`tests/index.js` importa también los módulos de aplicación que todavía no tienen escenarios, excepto `src/main.jsx`; así el reporte no oculta esas áreas al calcular cobertura. Abrir `coverage/index.html` tras ejecutar `npm test` y contrastar los archivos con menor cobertura con el alcance del informe. No afirmar “100% de calidad” ni “todos los errores están resueltos” porque una suite apruebe. Explicar las áreas pendientes o revisadas manualmente.

Preguntas posibles y puntos para responder

¿Por qué React?
Permite componer vistas reutilizables y representar la interfaz a partir de datos y estado. Aquí reduce la repetición de encabezados, tarjetas y formularios frente a las páginas independientes de la primera entrega.

¿Qué diferencia hay entre props y state?
Las props llegan desde el componente que utiliza al hijo. El estado representa valores que cambian durante la interacción. Se deben señalar ejemplos reales del proyecto, no solo dar definiciones.

¿Por qué usar contexto?
Productos, sesión y carrito se utilizan en varias rutas. El contexto y la suscripción evitan pasar esos datos manualmente por todos los niveles, mientras los filtros y formularios permanecen locales.

¿Qué significa CRUD en VESPER?
Crear, leer, actualizar y eliminar datos. Mostrar las operaciones de un mantenedor y cómo se reflejan en el catálogo y en el almacenamiento.

¿Dónde están la base de datos y el backend?
Esta entrega utiliza una fuente de datos JavaScript y persistencia local, como solicita la pauta. No hay servidor ni API proporcionados. El servicio concentra las operaciones para facilitar una futura integración, pero esa integración no se declara realizada.

¿La cuenta administradora es segura para producción?
No. Las credenciales y controles son de demostración en el navegador. Una solución real necesita autenticación, autorización y validación en el servidor, junto con almacenamiento seguro de contraseñas.

¿Qué pasa al cambiar de navegador?
Los datos locales no se comparten. Cada navegador y origen tiene su propia persistencia; limpiar los datos del sitio los elimina.

¿Por qué comprobar el stock en el servicio?
La validación visual puede omitirse o quedar desactualizada. Centralizar la regla permite aplicarla tanto al modificar el carrito como al confirmar la compra y probarla sin depender de una página específica.

¿Por qué el pedido conserva precios?
Para que un cambio posterior del catálogo no modifique el resumen histórico. El carrito utiliza precios vigentes; el pedido registra los de la compra aprobada.

¿Qué comprueba exactamente esta prueba?
Mostrar los datos iniciales, la acción y la aserción. Explicar qué defecto detectaría y qué queda fuera de su alcance.

¿Cuál es la diferencia entre prueba unitaria y revisión manual?
La prueba automatizada repite un escenario y verifica un resultado definido. La revisión manual permite explorar navegación, contenido y diseño. Las dos aportan evidencia distinta y se documentan por separado.

¿Cómo se usó IA?
Según la autorización del curso, se utilizó como apoyo al desarrollo. La defensa consiste en explicar, ejecutar y verificar la solución entregada. Cada integrante debe poder localizar las reglas, reproducir un flujo y explicar sus límites.

Ensayo individual

Cada integrante debe practicar, sin leer el guion, estas cuatro tareas: explicar un componente con props y estado; seguir una operación desde la interfaz hasta la persistencia; demostrar una prueba y su aserción; y reconocer una limitación de la simulación con la siguiente mejora técnica necesaria.